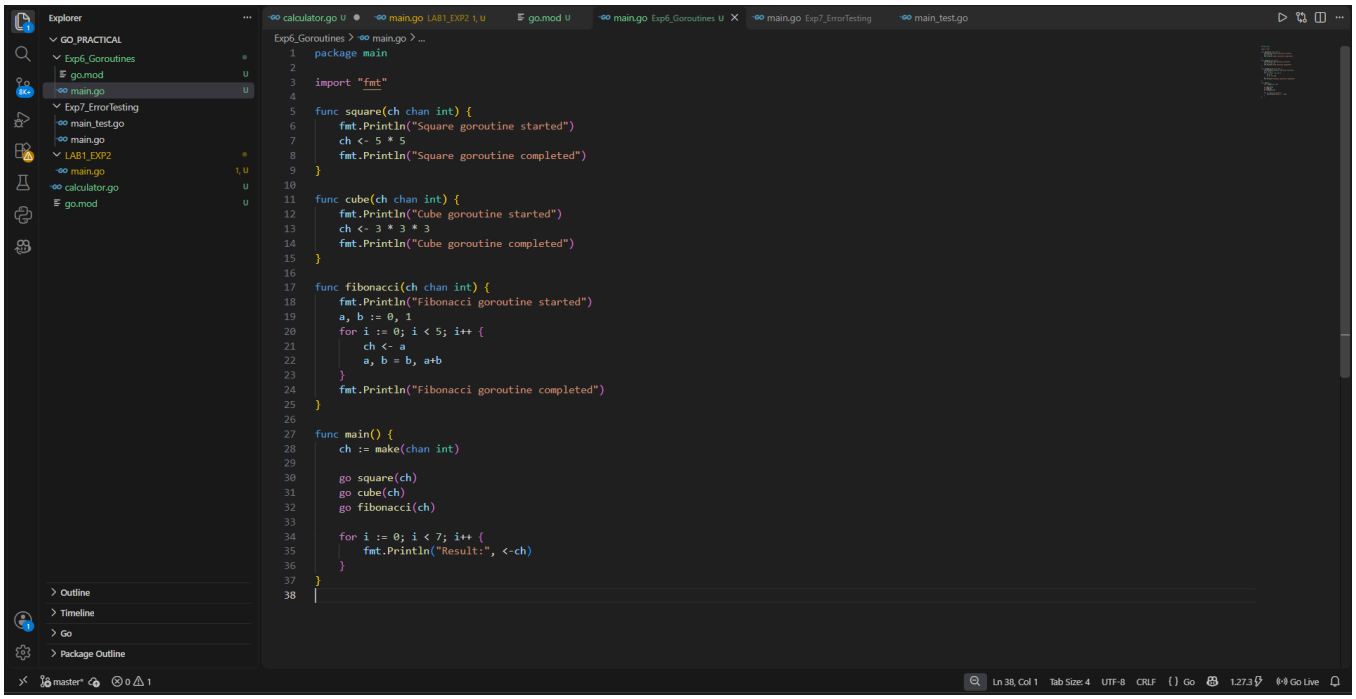
EXP 6

1. Concurrent Tasks Using Goroutines and Channels

    1. Write a Go program that launches at least three goroutines concurrently to perform independent tasks (e.g., computing squares, cubes, and Fibonacci numbers).
    2. Use a channel of the appropriate type to collect and print results in main().
    3. Add print statements showing goroutine start and completion to demonstrate concurrent execution order.



```
package main

import "fmt"

func square(ch chan int) {
    fmt.Println("Square goroutine started")
    ch <- 5 * 5
    fmt.Println("Square goroutine completed")
}

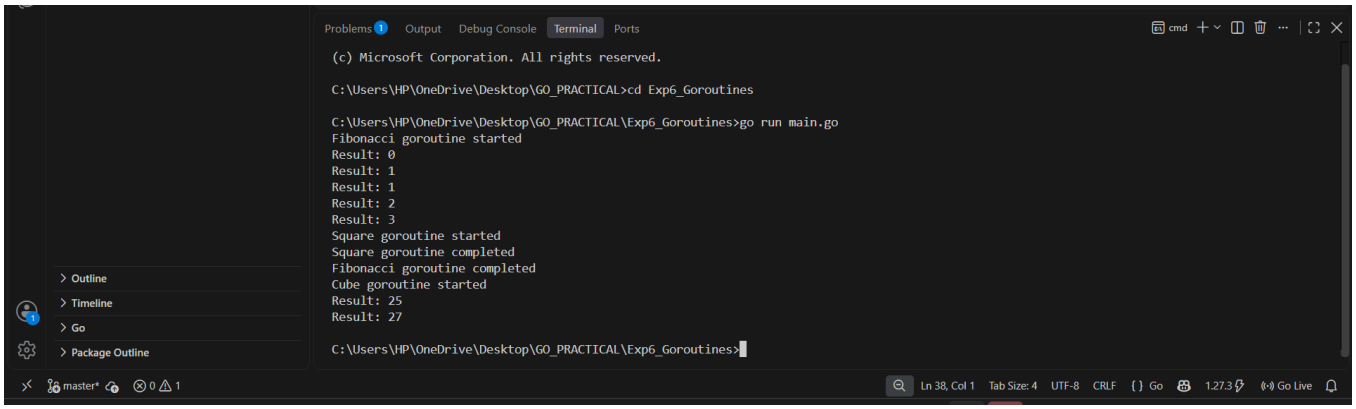
func cube(ch chan int) {
    fmt.Println("Cube goroutine started")
    ch <- 3 * 3 * 3
    fmt.Println("Cube goroutine completed")
}

func fibonacci(ch chan int) {
    fmt.Println("Fibonacci goroutine started")
    a, b := 0, 1
    for i := 0; i < 5; i++ {
        ch <- a
        a, b = b, a+b
    }
    fmt.Println("Fibonacci goroutine completed")
}

func main() {
    ch := make(chan int)

    go square(ch)
    go cube(ch)
    go fibonacci(ch)

    for i := 0; i < 7; i++ {
        fmt.Println("Result:", <-ch)
    }
}
```



```
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL>cd Exp6_Goroutines

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp6_Goroutines>go run main.go
Fibonacci goroutine started
Result: 0
Result: 1
Result: 1
Result: 2
Result: 3
Square goroutine started
Square goroutine completed
Fibonacci goroutine completed
Cube goroutine started
Result: 25
Result: 27

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp6_Goroutines>
```